Job

Цели работы:

- изучить концепцию Job в Kubernetes для управления одноразовыми задачами
- освоить создание и настройку Job с различными параметрами выполнения
- научиться управлять параллельным выполнением задач
- понять разницу между Job и Deployment
- освоить диагностику и Jobs

`Job` — ресурс Kubernetes для управления задачами, которые должны выполниться один или несколько раз и успешно завершиться. В отличие от Deployment, который поддерживает постоянно работающие приложения, Job предназначена для пакетной обработки, миграций данных и других одноразовых операций.

Пример 1: Простая Job

Базовая демонстрация работы Job для выполнения задачи, которая должна запуститься один раз и успешно завершиться.

Job создает Pod, который выполняет команду и завершается. `Параметр restartPolicy: Never` гарантирует, что Pod не будет перезапущен после успешного выполнения.

Шаг 1: Создаем Job для выполнения базовой задачи.

simple-job.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: batch/v1
kind: Job
metadata:
  name: simple-job
spec:
  template:
    spec:
      containers:
      - name: task
        image: busybox
        command: ['echo', 'Задача успешно выполнена!']
      restartPolicy: Never
```

Шаг 2: Проверяем выполнение Job.

- создаем Job
- отслеживаем статус
- проверяем логи

```
C:\Temp>kubectl apply -f simple-job.yaml
job.batch/simple-job created

C:\Temp>kubectl get jobs -w
NAME          STATUS     COMPLETIONS   DURATION   AGE
simple-job    Running    0/1           8s         8s
simple-job    Running    0/1           12s        12s
simple-job    Complete    1/1           12s        12s
```

```
C:\Temp>kubectl get pods
NAME                    READY   STATUS      RESTARTS        AGE
app-secret-env-pod      1/1     Running     1 (7d2h ago)    7d3h
app-secret-volume-pod   1/1     Running     1 (7d2h ago)    7d3h
simple-job-8fm6q        0/1     Completed   0               34s

C:\Temp>kubectl logs job/simple-job
Задача успешно выполнена!
```

Пример 2: Создание Job с обработкой ошибок и повторными попытками

Демонстрация отказоустойчивости Job при обработке сбоев с ограничением количества повторных попыток. Параметр `backoffLimit: 3` ограничивает количество повторных попыток при сбоях. `restartPolicy: OnFailure` указывает перезапускать Pod только при ошибках.

Шаг 1: Создаем Job с ограничением количества попыток

retry-job.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: retry-job
spec:
  backoffLimit: 3
  template:
    spec:
      containers:
      - name: processor
        image: busybox
        command: ['sh', '-c', 'echo "Попытка выполнения..."; exit 1']
      restartPolicy: OnFailure
```

Стр 13, стлб 31 100% Windows (CRLF) UTF-8

Шаг 2: Наблюдаем поведение при ошибках.

```
C:\Temp>kubectl apply -f retry-job.yaml
job.batch/retry-job created

C:\Temp>kubectl get pods -w
NAME                    READY   STATUS              RESTARTS        AGE
app-secret-env-pod      1/1     Running             1 (7d2h ago)    7d3h
app-secret-volume-pod   1/1     Running             1 (7d2h ago)    7d3h
retry-job-wpbrs         0/1     ContainerCreating   0               6s
simple-job-8fm6q        0/1     Completed           0               10m
retry-job-wpbrs         0/1     Error               0               6s
retry-job-wpbrs         0/1     Error               1 (3s ago)      9s
retry-job-wpbrs         0/1     CrashLoopBackOff    1 (1s ago)      10s
```

```
C:\Temp>kubectl describe job retry-job
Name:             retry-job
Namespace:        default
Selector:         batch.kubernetes.io/controller-uid=2690734d-9bc2-40cc-ae1f-697ea223ca7a
Labels:           batch.kubernetes.io/controller-uid=2690734d-9bc2-40cc-ae1f-697ea223ca7a
                  batch.kubernetes.io/job-name=retry-job
                  controller-uid=2690734d-9bc2-40cc-ae1f-697ea223ca7a
                  job-name=retry-job
Annotations:      <none>
Parallelism:      1
Completions:      1
Completion Mode:  NonIndexed
Suspend:          false
Backoff Limit:    3
Start Time:       Thu, 06 Nov 2024 22:44:43 +0300
Pods Statuses:    1 Active (0 Ready) / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  batch.kubernetes.io/controller-uid=2690734d-9bc2-40cc-ae1f-697ea223ca7a
           batch.kubernetes.io/job-name=retry-job
           controller-uid=2690734d-9bc2-40cc-ae1f-697ea223ca7a
           job-name=retry-job
```

Через несколько секунд: Pod завершится с ошибкой

```
Командная строка
                  job-name=retry-job
Annotations:      <none>
Parallelism:      1
Completions:      1
Completion Mode:  NonIndexed
Suspend:          false
Backoff Limit:    3
Start Time:       Thu, 06 Nov 2024 22:44:43 +0300
Pods Statuses:    0 Active (0 Ready) / 0 Succeeded / 1 Failed
Pod Template:
  Labels:  batch.kubernetes.io/controller-uid=2690734d-9bc2-40cc-ae1f-697ea223ca7a
           batch.kubernetes.io/job-name=retry-job
           controller-uid=2690734d-9bc2-40cc-ae1f-697ea223ca7a
           job-name=retry-job
```

```
Events:
  Type     Reason                Age    From            Message
  ----     ------                ----   ----            -------
  Normal   SuccessfulCreate      8m17s  job-controller  Created pod: retry-job-wpbrs
  Normal   SuccessfulDelete      7m21s  job-controller  Deleted pod: retry-job-wpbrs
  Warning  BackoffLimitExceeded  7m20s  job-controller  Job has reached the specified backoff limit
```

Пример 3: Создание Job с параллельным выполнением задач

Демонстрация распределенной обработки задач с управлением количеством одновременных выполнений. Параметр `completions: 6` задает общее количество успешных выполнений, `parallelism: 2` ограничивает количество одновременно работающих Pod'ов.

Шаг 1: Создаем Job для параллельной обработки.

parallel-job.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: batch/v1
kind: Job
metadata:
  name: parallel-job
spec:
  completions: 6
  parallelism: 2
  template:
    spec:
      containers:
      - name: worker
        image: busybox
        command: ['sh', '-c', 'echo "Обрабатываю задачу в Pod $HOSTNAME"; sleep 10']
      restartPolicy: Never
```

Шаг 2: Наблюдаем параллельное выполнение.

Ожидаемая последовательность событий:

1. Начальная фаза (сразу после создания Job): 2 Pod'а одновременно в статусе Running (параллелизм = 2)
2. Постепенное завершение Pod'ов: Первый Pod завершился (Completed). Сразу создался третий Pod (поддерживается параллелизм = 2)
3. Продолжение обработки: завершаются по очереди, создаются новые. Всегда 2 активных Pod'а (пока есть задачи)

```
C:\Temp>kubectl apply -f parallel-job.yaml
job.batch/parallel-job created

C:\Temp>kubectl get pods -w --selector=job-name=parallel-job
NAME                READY   STATUS              RESTARTS   AGE
parallel-job-g9bbg  0/1     ContainerCreating   0          7s
parallel-job-xllnb  1/1     Running             0          7s
parallel-job-g9bbg  1/1     Running             0          8s
parallel-job-xllnb  0/1     Completed           0          16s
parallel-job-xllnb  0/1     Completed           0          18s
parallel-job-96nnv  0/1     Pending             0          0s
parallel-job-96nnv  0/1     Pending             0          0s
parallel-job-xllnb  0/1     Completed           0          18s
parallel-job-96nnv  0/1     ContainerCreating   0          0s
parallel-job-g9bbg  0/1     Completed           0          19s
parallel-job-g9bbg  0/1     Completed           0          20s
parallel-job-hnr6n  0/1     Pending             0          0s
parallel-job-hnr6n  0/1     Pending             0          0s
parallel-job-g9bbg  0/1     Completed           0          21s
parallel-job-hnr6n  0/1     ContainerCreating   0          0s
```

4. Финальная фаза: все 6 Pod'ов завершены (completions = 6). Job выполнена успешно

```
parallel-job-rxbkl   0/1     Completed          0          14s
parallel-job-rxbkl   0/1     Completed          0          16s
parallel-job-rxbkl   0/1     Completed          0          16s
parallel-job-q8wx8   0/1     Completed          0          15s
parallel-job-q8wx8   0/1     Completed          0          17s
parallel-job-q8wx8   0/1     Completed          0          17s
```

Во время выполнения:

- `COMPLETIONS: N/6` - выполнено N задачи из 6,
- `DURATION: 15s` - прошло 15 секунд с начала выполнения

После завершения:

- `COMPLETIONS: 6/6` - все 6 задач успешно выполнены
- `DURATION: 54s` - общее время выполнения Job

```
C:\Temp>kubectl get job parallel-job -o wide
NAME           STATUS     COMPLETIONS   DURATION   AGE   CONTAINERS   IMAGES    SELECTOR
parallel-job   Complete   6/6           54s        94s   worker       busybox   batch.kubernetes.io/controller-ui
65ce4b-e9b1-4657-a5af-3431156bbdea
```

Пример 4: Создание Job для обработки данных с многострочным скриптом

Пример использования Job для обработки данных с сложной логикой выполнения. Многострочный скрипт позволяет выполнять сложные последовательности команд. Job гарантирует выполнение всей цепочки операций до успешного завершения

Шаг 1: Создаем Job для обработки файлов.

data-processing-job.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-processing-job
spec:
  template:
    spec:
      containers:
      - name: processor
        image: alpine
        command:
        - /bin/sh
        - -c
        - |
          echo "Начинаю обработку данных..."
          for i in 1 2 3 4 5; do
            echo "Обрабатываю элемент $i" >> /tmp/results.txt
            sleep 2
          done
          echo "Обработка завершена!" >> /tmp/results.txt
          cat /tmp/results.txt
      restartPolicy: OnFailure
```

Шаг 2: Проверяем результаты обработки.

```
C:\Temp>kubectl apply -f data-processing-job.yaml
job.batch/data-processing-job created

C:\Temp>kubectl wait --for=condition=complete job/data-processing-job --timeout=60s
job.batch/data-processing-job condition met

C:\Temp>kubectl logs job/data-processing-job
Начинаю обработку данных...
Обрабатываю элемент 1
Обрабатываю элемент 2
Обрабатываю элемент 3
Обрабатываю элемент 4
Обрабатываю элемент 5
Обработка завершена!

C:\Temp>kubectl get job data-processing-job
NAME                  STATUS     COMPLETIONS   DURATION   AGE
data-processing-job   Complete   1/1           17s        72s
```

Задания для самостоятельного решения

Задание 9.1. Базовая Job
Создайте Job с именем file-generator-job, которая:

- Создает файл со списком из 10 случайных чисел
- Сохраняет результат в /output/numbers.txt
- Завершает работу после успешного выполнения

Проверочные команды:

```
kubectl logs job/file-generator-job
kubectl get jobs -w
```

Задание 9.2. Job с параллельной обработкой
Создайте Job с именем batch-processor, которая:

- обрабатывает 8 задач
- запускает не более 3 задач одновременно
- каждая задача должна выводить свой уникальный идентификатор

Проверочные команды:

```
kubectl get pods --selector=job-name=batch-processor
kubectl describe job batch-processor
```

Задание 9.3. Job с обработкой ошибок
Создайте Job с именем fault-tolerant-job, которая:

- имитирует сбой в 50% случаев
- имеет максимум 4 попытки выполнения
- использует правильную политику перезапуска

Проверочные команды:

```
kubectl describe job fault-tolerant-job
kubectl get pods --selector=job-name=fault-tolerant-job | wc -l
```

Задание 9.4. Диагностика проблем с Job
Создайте Job с именем problematic-job, которая не может завершиться успешно:

- проанализируйте причины сбоя
- изучите логи и события
- предложите решение проблемы

Проверочные команды:

```
kubectl logs job/problematic-job
kubectl describe job problematic-job
kubectl get events --field-selector involvedObject.name=problematic-job
```

Контрольные вопросы

1. В чем основное отличие Job от Deployment?
2. Какие значения restartPolicy допустимы для Job и в чем их разница?
3. Что происходит с Pod'ами после успешного завершения Job?
4. Как параметр backoffLimit влияет на поведение Job при ошибках?
5. В чем разница между parallelism и completions?
6. Как можно посмотреть логи завершенной Job?
7. Что произойдет, если Job превысит установленный backoffLimit?
8. Какие команды вы бы использовали для диагностики проблем с Job?